Campus-Maintenance-System-C

A colorful, terminal-based Campus Maintenance Management System built in C, featuring persistent file I/O and report generation.

By Cynthia

12/2025

Contents

1 Introduction & Program Design 1

1.1 Overview 1

1.2 Data Structures 1

2 Main Features & Implementation 2

2.1 Input Handling 2

2.2 Status Update 2

2.3 Search Functionality 2

3 Flowchart: Find Technician by ID 2

4 Testing and Results 3

4.1 Core Functionality & Search 3

4.2 Error Handling & Input Validation 5

4.3 Logical Workflow Enforcement 6

4.4 File Output & Reporting 7

5 Creativity & Extension Features 8

5.1 Real-Time Statistics Dashboard 8

5.2 Data Persistence 9

5.3 Specialty Verification 10

5.4 UI Design 10

6 Conclusion 10

Appendix A: Source code 10

1 Introduction & Program Design

1.1 Overview

This report presents a Campus Maintenance Management System developed using C language. The main goal is to create a practical tool that tracks campus equipment maintenance issues from start to finish. Unlike a simple data logging program, this system attempts to simulate the operational logic of real-world software.

Specifically, there are three features in this project: first, error handling and recovery mechanisms is achieved to deal with various complex situations. Second, full database functionality, including adding, deleting, modifying, and searching data, was achieved. Finally, a user interface that is both aesthetic and concise is designed.

1.2 Data Structures

For data storage, the whole code uses structs to describe the data, allowing different data types to be stored in a single object. Inside the Request struct, there are integer values like reqId (the unique identification code for each request), priority, and techId (which links to the Technician struct). It also stores strings for details such as location, category, and description. All these pieces of information combined make up a complete request object.

```c
struct Request {
    int reqId;              // Unique number for each request(no duplicates)[Page 3, 1.1]
    char location[50];      // Where the issue is
    char category[20];      // Type of problem
    int priority;           // Urgency 1-5 (high to low) [Page 3, 1.1]
    char status[12];        // "NEW", "ASSIGNED", "DONE" [Page 3, 1.1]
    int techId;             // Assigned technician ID, -1 if none [Page 3, 1.1]
};
```

struct Request: The core structure stores all job details, such as the unique ID, location, priority, and status. The system uses a text string for the status to track the job's progress. The status options are 'NEW', 'ASSIGNED', and 'DONE'.

```c
struct Technician {
    int techId;              // Unique ID [Page 3, 1.2]
    char name[60];           // Full name
    char specialty[20];      // Main skill
};
```

struct Technician: The core structure holding the technician's profile, including their techId, name, and specialty.

2 Main Features & Implementation

This chapter will provide a detailed account of the principal implementation of project.

2.1 Input Handling

The original scanf function causes problems. Typing a letter instead of a number crashes the program or starts an infinite loop. To fix this, the code adds a validation check. Also, a clearInputBuffer() function handles the Enter key so the program reads later inputs correctly.

```
// 3. [Page 3, 2.1]
    do {
        printf("Enter Priority (1-High to 5-Low): ");
        if (scanf("%d", &newReq.priority) != 1) {
            clearInputBuffer();
            newReq.priority = 0; // Force loop
        } else clearInputBuffer();

        if (newReq.priority < 1 || newReq.priority > 5)
            printf(COLOR_RED "Invalid priority! Use 1-5.\n" COLOR_RESET);
} while (newReq.priority < 1 || newReq.priority > 5);
```

2.2 Status Update

To forcefully ensure the technician receives the assignment before the request is completed, the code should prevent changing a request directly from NEW to DONE. Otherwise, the scenario would be illogical in real world. So, the updateStatus function includes a logic check, verifies if the status change is valid and stops any attempt to finish a request directly without the proper steps.

2.3 Search Functionality

A linear search finds specific IDs to help get data. It looks through the array to find the target. If the search finds a match, the function returns the index. If not, it returns -1 to show failure. The findRequestId and findTechnicianId functions use this logic, and the "Assign" and "Update" modules depend on it.

3 Flowchart: Find Technician by ID

The flowchart shows the search logic for technician records. The algorithm checks the data list one by one and compares the target ID with each item. If a match occurs, the system gives back the index. Otherwise, it returns -1 to show that the record does not exist.

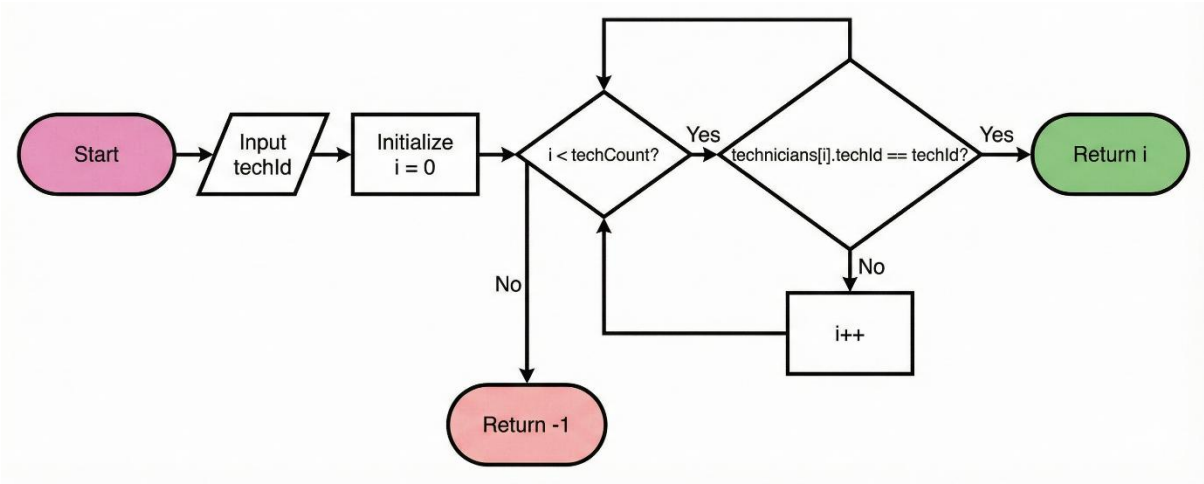


Figure 1: Flowchart

4 Testing and Results

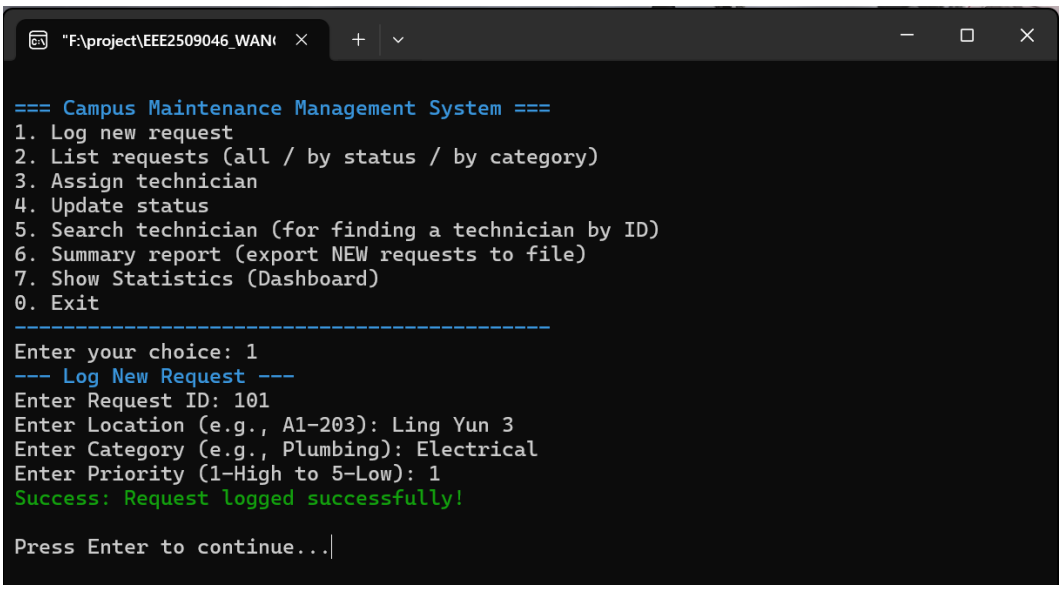
The code undergoes strict testing to prevent errors. This process happens in two stages. First, the system processes normal inputs. Second, it handles bad inputs to test if the program breaks. The screenshots below show how the system work.

4.1 Core Functionality & Search

In order to verify the basic actions, a simulation was conducted of a complete process of a maintenance request.

1. **Logging & Assignment**

   Firstly, I logged a new request ID101 and then assigned it to Technician 101. The board now showed me that my request has been successfully loaded.



```
Enter your choice: 3
--- Assign Technician ---
Enter Request ID: 101
Enter Technician ID (Available: 101, 102, 103): 101
Success: Tech Amy assigned to Req 101.

Press Enter to continue...
```

Figure 2: Send an Assignment Request

2.Status Update

I selected 2. List requests to check that request ID101 is currently in the status of "ASSIGNED". Next, I chose 4. Update status to ensure my program is working correctly, and Figure 3 shows that I successfully changed the status of request ID101 from "ASSIGNED" to "DONE".

```
"F:\project\EEE2509046_WAN(

=== Campus Maintenance Management System ===
1. Log new request
2. List requests (all / by status / by category)
3. Assign technician
4. Update status
5. Search technician (for finding a technician by ID)
6. Summary report (export NEW requests to file)
7. Show Statistics (Dashboard)
0. Exit
--------------------------------------------
Enter your choice: 2

--- List Options ---
1. List all
2. Filter by status
3. Filter by category
Enter option: 1

ID    Location            Category        Pri      Status       TechID
--------------------------------------------------------------------------
101   Ling Yun 3          Electrical      1        ASSIGNED     101

Press Enter to continue...
--------------------------------------------
Enter your choice: 4
--- Update Status ---
Enter Request ID: 101

Request Details:
  ID: 101 | Location: Ling Yun 3 | Category: Electrical
  Current Technician ID: 101
  Status Workflow: NEW -> [ASSIGNED] -> DONE
--------------------------------------------
This request is currently assigned to Technician ID 101.
Is the maintenance job fully completed? (y/n): y
Success: Status updated to 'DONE'. Job Closed.
```

Figure 3: Switch the Status Workflow.

3.Search

I entered choice 5 to search the technician 103 and the board showed me all the correct information of the technician.

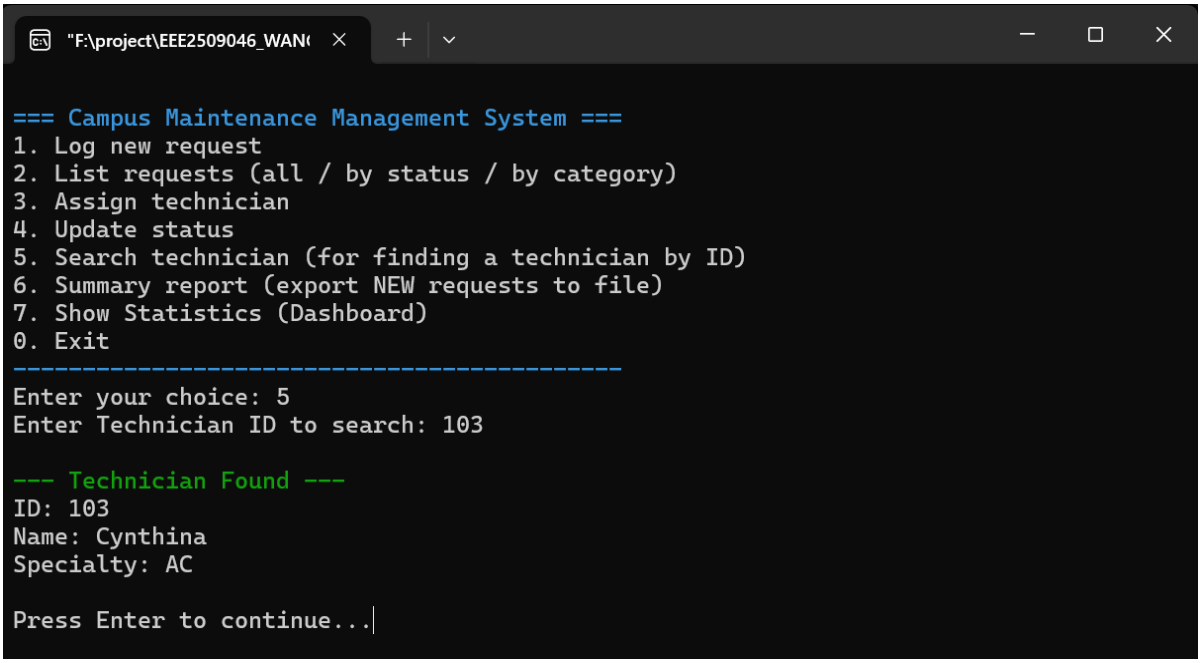


Figure 4: Send a Query Request

4.2 Error Handling & Input Validation

To verify the integrity of the error handling mechanisms, negative testing by intentionally injecting invalid data inputs is performed.

1. **Duplicate ID Check**

I entered the same request ID again, and the system reported an error.

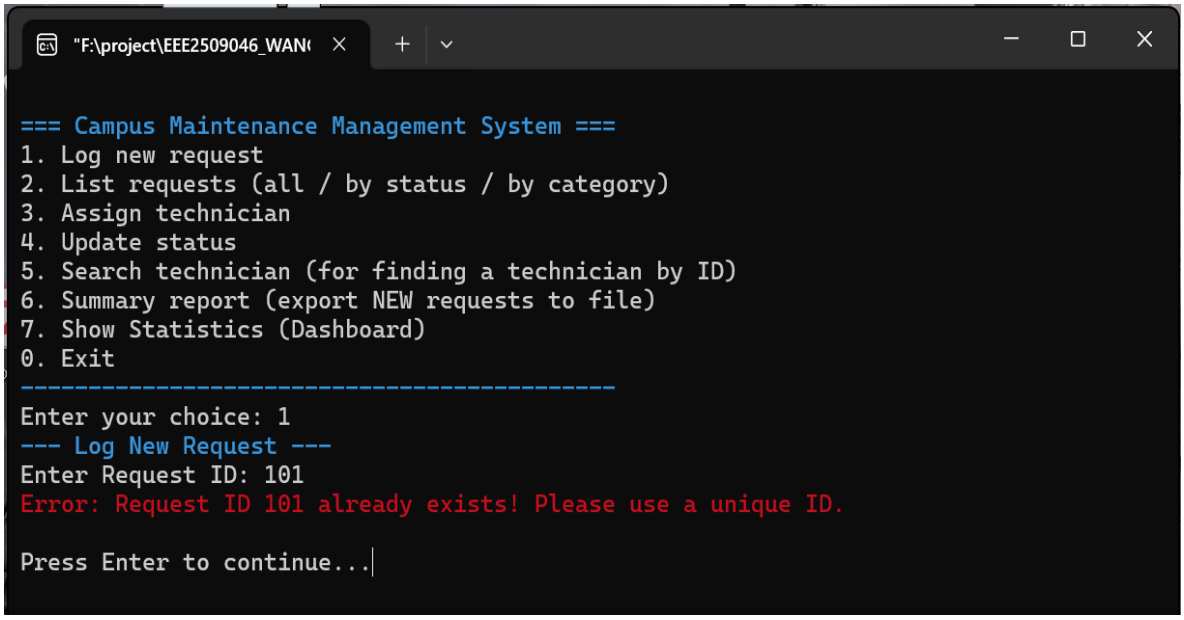


Figure 5: Error Notification

2. **Range Check**

I entered a wrong priority 9, and the system reported an error.

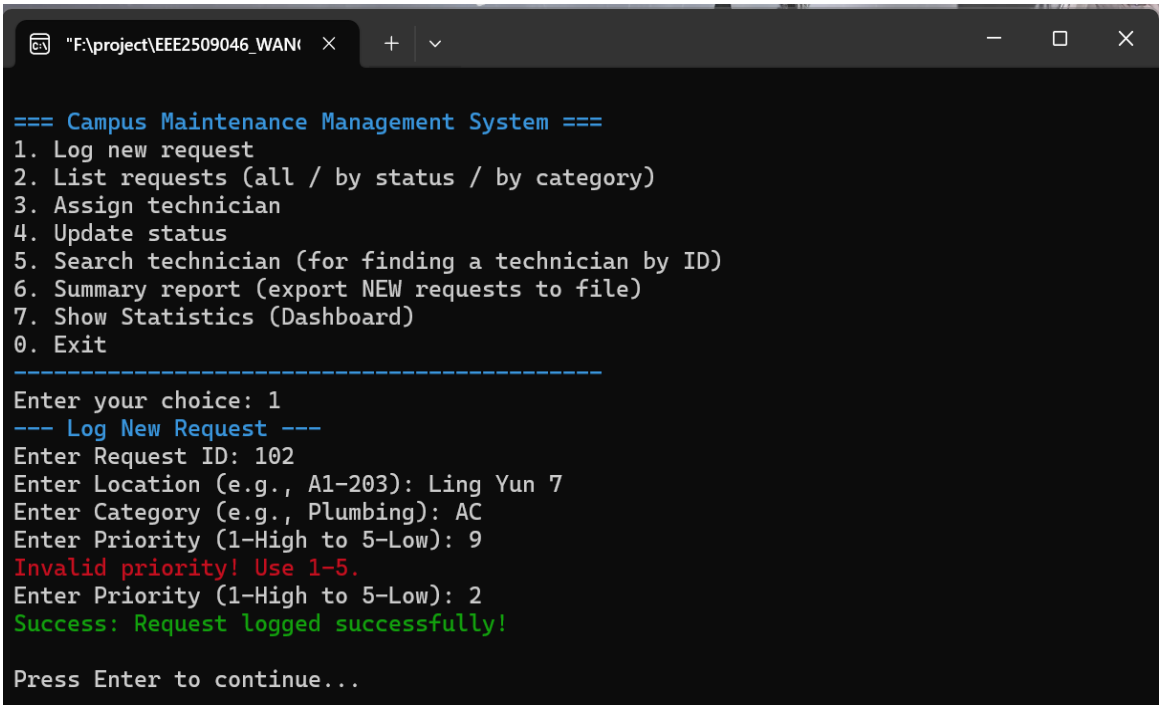


Figure 6: Breakdown for Out-of-Range Action

4.3 Logical Workflow Enforcement

To check the workflow order, a test tries to change a request status straight from "NEW" to "DONE." The system rejects this action correctly, which confirms that the rules work.

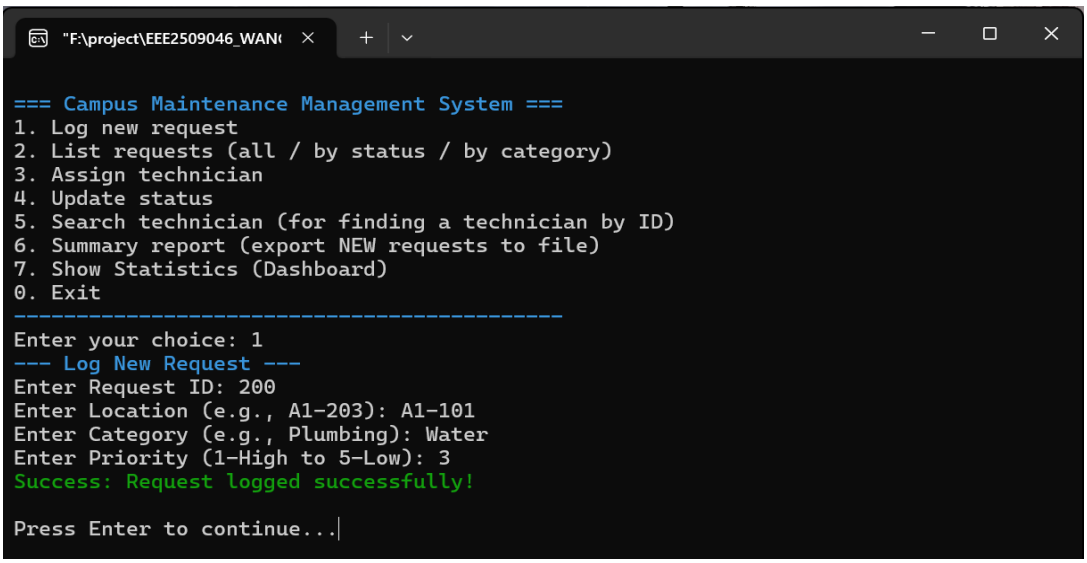


Figure 7: Logging Request ID 200.

```
"F:\project\EEE2509046_WAN(
=== Campus Maintenance Management System ===
1. Log new request
2. List requests (all / by status / by category)
3. Assign technician
4. Update status
5. Search technician (for finding a technician by ID)
6. Summary report (export NEW requests to file)
7. Show Statistics (Dashboard)
0. Exit
--------------------------------------------
Enter your choice: 4
--- Update Status ---
Enter Request ID: 200

Request Details:
  ID: 200 | Location: A1-101 | Category: Water
  Current Technician ID: -1
  Status Workflow: [NEW] -> ASSIGNED -> DONE
--------------------------------------------
Action Denied: Current status is 'NEW'.
Explanation: You cannot skip the 'ASSIGNED' stage.
Instruction: Please use Option 3 (Assign Technician) to assign a technician first.

Press Enter to continue...
```

Figure 8: Blocking the Stage Skipping

4.4 File Output & Reporting

Option 6 creates a summary file to test report generation. Checking the output file, new_requests.txt, shows that the filter works correctly. The file keeps only requests with a "NEW" status (IDs 102 and 200) and leaves out completed tickets.

```
"F:\project\EEE2509046_WAN(

=== Campus Maintenance Management System ===
1. Log new request
2. List requests (all / by status / by category)
3. Assign technician
4. Update status
5. Search technician (for finding a technician by ID)
6. Summary report (export NEW requests to file)
7. Show Statistics (Dashboard)
0. Exit
--------------------------------------------
Enter your choice: 6

--- Generating Summary Report ---
Scanning database for 'NEW' requests...
 -> Exporting Request ID: 102 (Loc: Ling Yun 7)
 -> Exporting Request ID: 200 (Loc: A1-101)
--------------------------------------------
Success: Report generation complete.
Total Records Exported: 2
Data saved to file: new_requests.txt
You can find this file in the program directory.

Press Enter to continue...
```

Figure 9: Successful Exporting Action

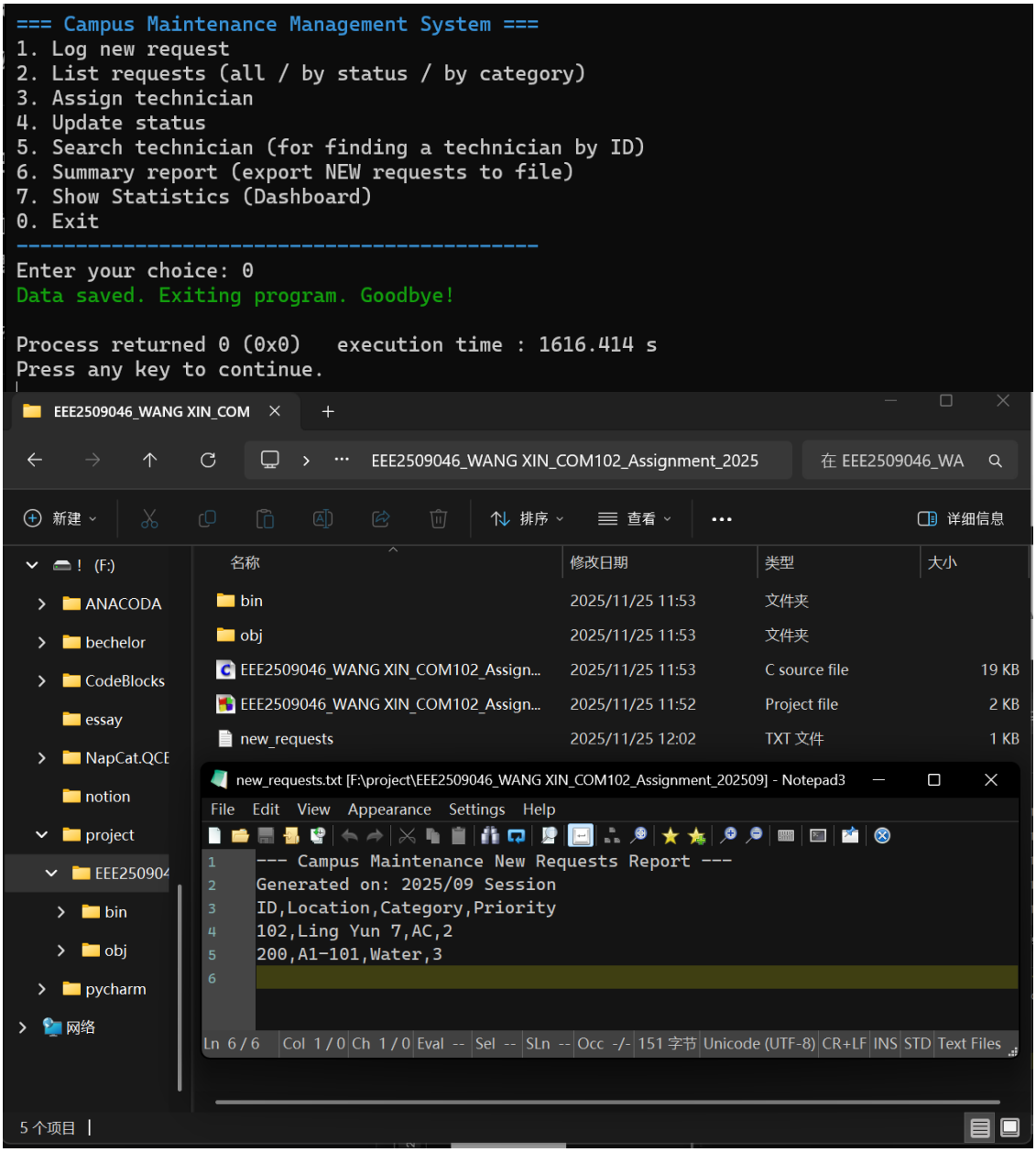


Figure 10: Saved Data File

5 Creativity & Extension Features

Beyond the basic functional specifications, extra enhancements were implemented to optimize the usability of system and operational efficiency.

5.1 Real-Time Statistics Dashboard

If the screen just lists the status of every request, it would not be very intuitive for the user and makes it hard to count the numbers. So, the program has a Dashboard function in Option 7. When it is selected, this function loops through all the requests stored in the system, checks their current status, and then displays the totals in three categories.

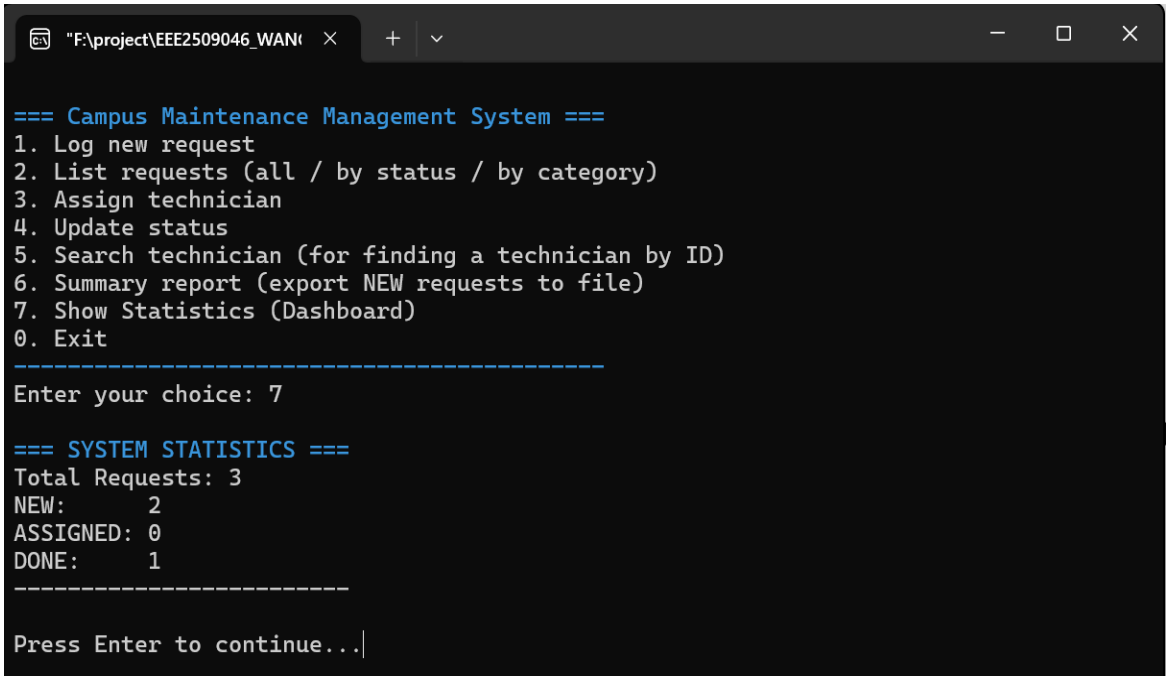


Figure 11: Dashboard.

5.2 Data Persistence

Since the program stores data in memory while running, I needed a way to save it permanently. This is done by saving to a file. When the program closes, it runs a function to loop through all the data. It formats the data and saves it into all_data_db.txt. On the other hand, when the program starts, it runs a reading function. It checks if the data file exists. If it does, the function reads the file and restores the data to its previous state.

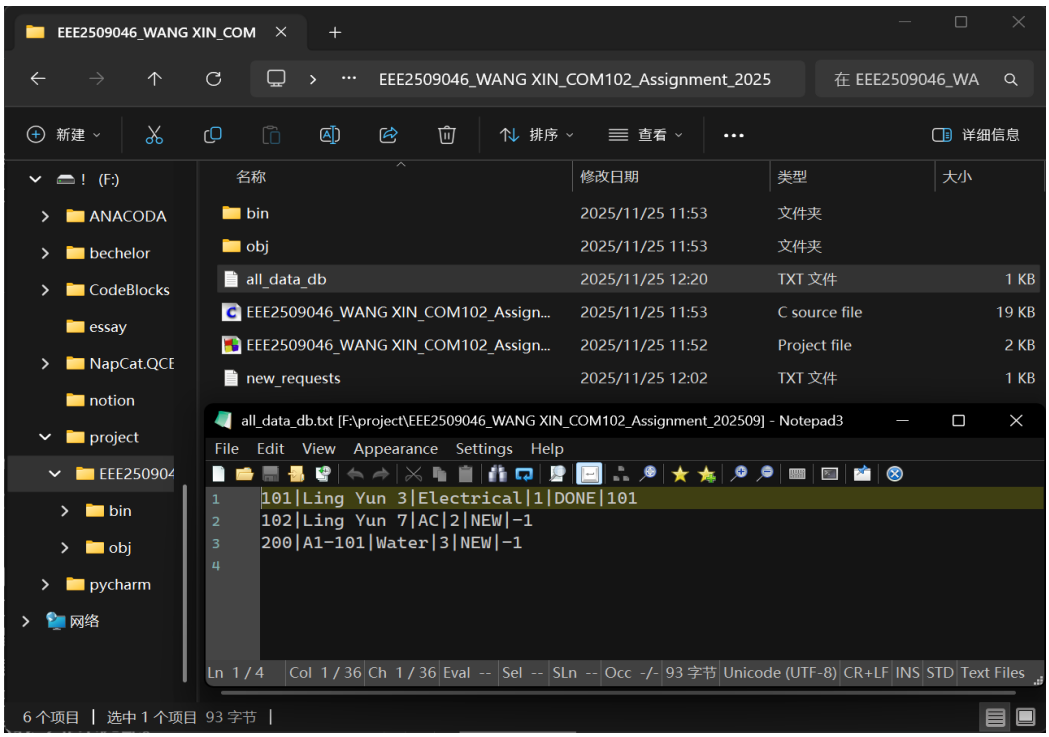


Figure 12: Auto-Saved Data File

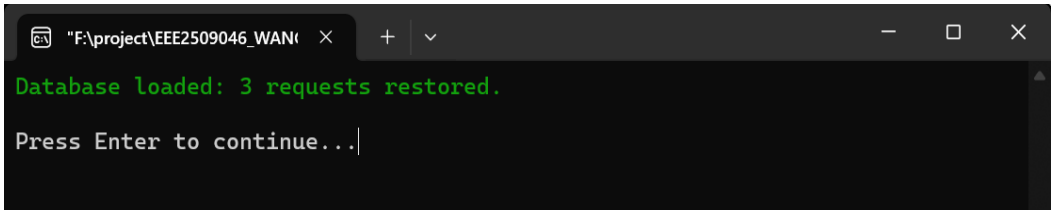


Figure 13: Auto-Restoration

5.3 Specialty Verification

A check ensures the system assigns resources correctly. This function compares the request category against the technician's skill.

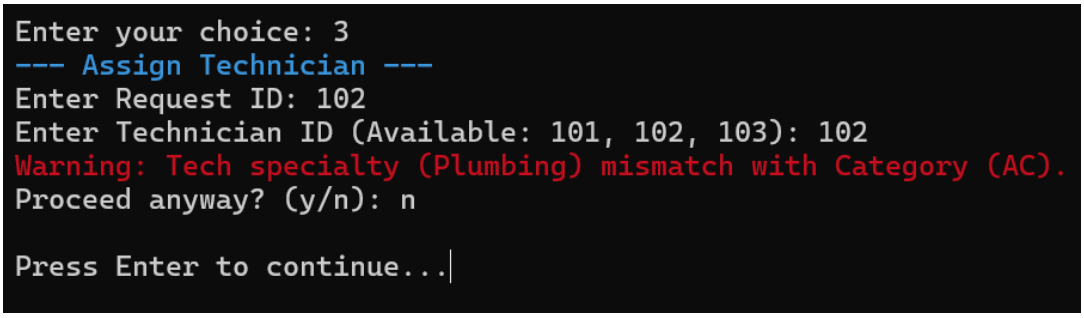


Figure 14: Smart Check

5.4 UI Design

To enhance interface usability, ANSI escape sequences is integrated to establish a clear visual hierarchy. Semantic coloring is applied to system outputs, rendering error states in Red, confirmation messages in Green, and section headers in Cyan. This visual encoding facilitates immediate status recognition.

6 Conclusion

Building this system helps give a better understanding of the C language. Connecting the functions presents a challenge, especially regarding how arrays move data. After hard work, a stable system runs. The extra features, such as saving data and using colors, create a sense of success, making the software feel like a real tool rather than just a school project.

Appendix A: Source code

```
/*
   Project: Campus Maintenance Management System
   Author: WANG XIN
   Student ID: EEE2509046
```

```c
*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Defines max capacity
#define MAX_REQUESTS 500
#define MAX_TECHNICIANS 50
#define FILENAME "new_requests.txt"      // [Page 4, 2.6]
#define DB_FILENAME "all_data_db.txt"

// ANSI Color Codes for Better UI (Creativity)
#define COLOR_RED     "\x1b[31m"
#define COLOR_GREEN   "\x1b[32m"
#define COLOR_CYAN    "\x1b[36m"
#define COLOR_RESET   "\x1b[0m"

// 1. Data Structures [Page 3, Section 1]

struct Request {
    int reqId;          // Unique number for each request(no duplicates)[Page 3, 1.1]
    char location[50];     // Where the issue is
    char category[20];     // Type of problem
    int priority;          // Urgency 1-5 (high to low) [Page 3, 1.1]
    char status[12];       // "NEW", "ASSIGNED", "DONE" [Page 3, 1.1]
    int techId;            // Assigned technician ID, -1 if none [Page 3, 1.1]
};

struct Technician {
    int techId;             // Unique ID [Page 3, 1.2]
    char name[60];          // Full name
    char specialty[20];     // Main skill
};


struct Request requests[MAX_REQUESTS];
int requestCount = 0;

struct Technician technicians[MAX_TECHNICIANS];
int techCount = 0;

void initTechnicians();
void loadData();
void saveData();
void logNewRequest();             // [Page 3, 2.1]
void listRequests();              // [Page 4, 2.2]
void assignTechnician();          // [Page 4, 2.3]
void updateStatus();              // [Page 4, 2.4]
```

```
void searchTechnician();          // [Page 4, 2.5]
void generateReport();            // [Page 4, 2.6]
void showStatistics();
int findRequestById(int id);      // [Page 5, 3.1]
int findTechnicianById(int id);   // [Page 5, 3.2]
//Help Function
void clearInputBuffer();
void pauseAndClear();                 //UI Helper
void getLine(char *buffer, int size); //Input Helper

// Main Function [Page 5, 4. Menu-Driven Interface]
int main() {
    // 1. Load existing data if available (Persistence) [Page 4, 2.3 check point]
    initTechnicians();
    loadData();

    int choice;
    do {
            // Menu UI
            system("cls"); // Clear screen for Windows
            printf(COLOR_CYAN "\n=== Campus Maintenance Management System ===\n"
COLOR_RESET);
            printf("1. Log new request\n");
            printf("2. List requests (all / by status / by category)\n");
            printf("3. Assign technician\n");
            printf("4. Update status\n");
            printf("5. Search technician (for finding a technician by ID)\n");
            printf("6. Summary report (export NEW requests to file)\n");
            printf("7. Show Statistics (Dashboard)\n");  // Extension
            printf("0. Exit\n");
            printf(COLOR_CYAN "--------------------------------------------\n"
COLOR_RESET);
            printf("Enter your choice: ");

        // Simple input validation to prevent entering letters that could lead to an
infinite loop
        if (scanf("%d", &choice) != 1) {
                printf(COLOR_RED "Invalid input! Please enter a number.\n" COLOR_RESET);
                clearInputBuffer();
                pauseAndClear();
                continue;
        }
        clearInputBuffer(); // Delete the line break

        switch(choice) {
              case 1: logNewRequest(); pauseAndClear(); break;
              case 2: listRequests(); pauseAndClear(); break;
              case 3: assignTechnician(); pauseAndClear(); break;
```

```
            case 4: updateStatus(); pauseAndClear(); break;
            case 5: searchTechnician(); pauseAndClear(); break;
            case 6: generateReport(); pauseAndClear(); break;
            case 7: showStatistics(); pauseAndClear(); break;
            case 0:
                saveData(); // Save before exit
                printf(COLOR_GREEN "Data saved. Exiting program. Goodbye!\n"
COLOR_RESET);
                break;
            default:
                printf(COLOR_RED "Invalid choice! Please try again.\n" COLOR_RESET);
                pauseAndClear();
        }
    } while (choice != 0);

    return 0;
}

// Helper Functions

void clearInputBuffer() { // Clear the input buffer to prevent scanf from skipping input
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}

void pauseAndClear() { // UI Helper: Pauses screen so user can read result, then clears
    printf("\nPress Enter to continue...");
    getchar(); // Wait for user
}

void getLine(char *buffer, int size) {// Input Helper: Safer string input than scanf or raw
fgets
    if (fgets(buffer, size, stdin) != NULL) {
        size_t len = strlen(buffer);
        if (len > 0 && buffer[len-1] == '\n') {
            buffer[len-1] = '\0';
        }
    }
}

// Initialize [Page 4, 2.3]
void initTechnicians() {
    technicians[0].techId = 101;
    strcpy(technicians[0].name, "Amy");
    strcpy(technicians[0].specialty, "Electrical");

    technicians[1].techId = 102;
    strcpy(technicians[1].name, "Bob");
```

```c
    strcpy(technicians[1].specialty, "Plumbing");

    technicians[2].techId = 103;
    strcpy(technicians[2].name, "Cynthina");
    strcpy(technicians[2].specialty, "AC");

    techCount = 3;
}

// [Page 5, 3.1] Search function - findRequestById
// Return the index of the matching request if found. Return -1 if no request with that ID
exists.
int findRequestById(int id) {
    for (int i = 0; i < requestCount; i++) {
        if (requests[i].reqId == id) return i;
    }
    return -1;
}

// [Page 5, 3.2] Search function - findTechnicianById
// Return the index of the matching technician if found. Return -1 if no technician with
that ID exists.
// Flowchart
int findTechnicianById(int id) {
    for (int i = 0; i < techCount; i++) {
        if (technicians[i].techId == id) return i;
    }
    return -1;
}

// Core Features

void loadData() {  // Load data from file at startup
    FILE *fp = fopen(DB_FILENAME, "r");
    if (fp == NULL) return; // File doesn't exist yet, that's fine

    // File format: ID|Location|Category|Priority|Status|TechID
    while (fscanf(fp, "%d|%49[^|]|%19[^|]|%d|%11[^|]|%d\n",
           &requests[requestCount].reqId,
           requests[requestCount].location,
           requests[requestCount].category,
           &requests[requestCount].priority,
           requests[requestCount].status,
           &requests[requestCount].techId) == 6) {
        requestCount++;
    }
    fclose(fp);
```

```
    printf(COLOR_GREEN "Database loaded: %d requests restored.\n" COLOR_RESET,
requestCount);
    pauseAndClear();
}

void saveData() { // Save ALL data to file on exit
    FILE *fp = fopen(DB_FILENAME, "w");
    if (fp == NULL) {
        printf(COLOR_RED "Error: Could not save data!\n" COLOR_RESET);
        return;
    }
    for (int i = 0; i < requestCount; i++) {
        fprintf(fp, "%d|%s|%s|%d|%s|%d\n",
                requests[i].reqId,
                requests[i].location,
                requests[i].category,
                requests[i].priority,
                requests[i].status,
                requests[i].techId);
    }
    fclose(fp);
}

// [Page 3, 2.1] Log a New Request
void logNewRequest() {
    if (requestCount >= MAX_REQUESTS) {
        printf(COLOR_RED "Error: Database full!\n" COLOR_RESET);
        return;
    }

    struct Request newReq;
    printf(COLOR_CYAN "--- Log New Request ---\n" COLOR_RESET);

    // 1. scanf and check ID [Page 5, 3.3: Reject duplicate reqId]
    printf("Enter Request ID: ");

    if (scanf("%d", &newReq.reqId) != 1) { // Input Validation
        printf(COLOR_RED "Error: Invalid input! A numeric ID is required.\n" COLOR_RESET);
        clearInputBuffer();
        return;
    }
    clearInputBuffer();

    if (findRequestById(newReq.reqId) != -1) {    // Duplicate Check
        printf(COLOR_RED "Error: Request ID %d already exists! Please use a unique ID.\n"
COLOR_RESET, newReq.reqId);
        return;
    }
```

```
    // 2. Input details with Empty Check
    do {
        printf("Enter Location (e.g., A1-203): ");
        getLine(newReq.location, 50);
        if (strlen(newReq.location) == 0) printf(COLOR_RED "Location cannot be empty.\n"
COLOR_RESET);
    } while (strlen(newReq.location) == 0);

    do {
        printf("Enter Category (e.g., Plumbing): ");
        getLine(newReq.category, 20);
        if (strlen(newReq.category) == 0) printf(COLOR_RED "Category cannot be empty.\n"
COLOR_RESET);
    } while (strlen(newReq.category) == 0);

    // 3. [Page 3, 2.1]
    do {
        printf("Enter Priority (1-High to 5-Low): ");
        if (scanf("%d", &newReq.priority) != 1) {
            clearInputBuffer();
            newReq.priority = 0; // Force loop
        } else clearInputBuffer();

        if (newReq.priority < 1 || newReq.priority > 5)
            printf(COLOR_RED "Invalid priority! Use 1-5.\n" COLOR_RESET);
    } while (newReq.priority < 1 || newReq.priority > 5);

    // 4. [Page 3, 2.1: Set status = "NEW" and techId = -1 if not assigned.]
    strcpy(newReq.status, "NEW");
    newReq.techId = -1;

    // save in array
    requests[requestCount] = newReq;
    requestCount++;

    printf(COLOR_GREEN "Success: Request logged successfully!\n" COLOR_RESET);
}

// [Page 4, 2.2] List Requests
void listRequests() {
    if (requestCount == 0) {
        printf(COLOR_RED "No requests found.\n" COLOR_RESET);
        return;
    }

    int option;
    printf("\n--- List Options ---\n");
```

```
    printf("1. List all\n2. Filter by status\n3. Filter by category\n");
    printf("Enter option: ");
    scanf("%d", &option);
    clearInputBuffer();

    char filter[20];
    if (option == 2) {
        printf("Enter status (NEW/ASSIGNED/DONE): ");
        getLine(filter, 20);
    } else if (option == 3) {
        printf("Enter Category: ");
        getLine(filter, 20);
    }
    //output the table
    printf(COLOR_CYAN "\n%-5s %-20s %-15s %-8s %-12s %-8s\n", "ID", "Location", "Category",
"Pri", "Status", "TechID");
    printf("-----------------------------------------------------------------------------\n"
COLOR_RESET);

    int found = 0;
    for (int i = 0; i < requestCount; i++) {
        int match = 0;
        if (option == 1) match = 1; // All
        else if (option == 2 && strcmp(requests[i].status, filter) == 0) match = 1; // By
Status
        else if (option == 3 && strcmp(requests[i].category, filter) == 0) match = 1; // By
Category

        if (match) {
            printf("%-5d %-20s %-15s %-8d %-12s %-8d\n",
                   requests[i].reqId,
                   requests[i].location,
                   requests[i].category,
                   requests[i].priority,
                   requests[i].status,
                   requests[i].techId);
            found = 1;
        }
    }
    if (!found) printf("No matching requests found.\n"); // [Page 4, 2.2: If no match
found, print message]
}

// [Page 4, 2.3] Assign Technician
void assignTechnician() {
    int reqId, techId;
    printf(COLOR_CYAN "--- Assign Technician ---\n" COLOR_RESET);
    printf("Enter Request ID: ");
```

```c
    scanf("%d", &reqId);
    clearInputBuffer();

    // Check whether the request exists
    int rIndex = findRequestById(reqId);
    if (rIndex == -1) {
        printf(COLOR_RED "Error: Request ID not found.\n" COLOR_RESET);
        return;
    }

    // Check the current status. If it has already been completed, do not allocate it again
    if (strcmp(requests[rIndex].status, "DONE") == 0) {
        printf(COLOR_RED "Error: This request is already DONE.\n" COLOR_RESET);
        return;
    }

    printf("Enter Technician ID (Available: 101, 102, 103): ");
    scanf("%d", &techId);
    clearInputBuffer();

    // [Page 4, 2.3] Check that technician exists
    int tIndex = findTechnicianById(techId);
    if (tIndex == -1) {
        printf(COLOR_RED "Error: Technician ID not found.\n" COLOR_RESET);
        return;
    }

    // [Page 4, 2.3] Optional: Check specialty matches category
    // Prompt the user if their specialty does not matched
    if (strstr(requests[rIndex].category, technicians[tIndex].specialty) == NULL) {
        printf(COLOR_RED "Warning: Tech specialty (%s) mismatch with Category (%s).\n"
COLOR_RESET,
               technicians[tIndex].specialty, requests[rIndex].category);
        char confirm;
        printf("Proceed anyway? (y/n): ");
        scanf("%c", &confirm);
        clearInputBuffer();
        if (confirm != 'y' && confirm != 'Y') return;
    }

    // [Page 4, 2.3] If both exist: Set request.techId and status = "ASSIGNED"
    requests[rIndex].techId = techId;
    strcpy(requests[rIndex].status, "ASSIGNED");
    printf(COLOR_GREEN "Success: Tech %s assigned to Req %d.\n" COLOR_RESET,
           technicians[tIndex].name, reqId);
}

// [Page 4, 2.4] Update Status
```

```c
void updateStatus() {
    int reqId;
    printf(COLOR_CYAN "--- Update Status ---\n" COLOR_RESET);
    printf("Enter Request ID: ");
    scanf("%d", &reqId);
    clearInputBuffer();

    int idx = findRequestById(reqId);
    if (idx == -1) {
        printf(COLOR_RED "Error: Request ID %d not found in database.\n" COLOR_RESET,
reqId);
        return;
    }

    // Visualizing the workflow (Creativity)
    printf("\nRequest Details:\n");
    printf("  ID: %d | Location: %s | Category: %s\n",
           requests[idx].reqId, requests[idx].location, requests[idx].category);
    printf("  Current Technician ID: %d\n", requests[idx].techId);

    printf("  Status Workflow: ");
    if (strcmp(requests[idx].status, "NEW") == 0)
        printf(COLOR_RED "[NEW]" COLOR_RESET " -> ASSIGNED -> DONE\n");
    else if (strcmp(requests[idx].status, "ASSIGNED") == 0)
        printf("NEW -> " COLOR_CYAN "[ASSIGNED]" COLOR_RESET " -> DONE\n");
    else
        printf("NEW -> ASSIGNED -> " COLOR_GREEN "[DONE]" COLOR_RESET "\n");
    printf(COLOR_CYAN "-------------------------------------------\n" COLOR_RESET);

    // [Page 4, 2.4] Allow only valid transitions NEW -> ASSIGNED ASSIGNED -> DONE
    if (strcmp(requests[idx].status, "NEW") == 0) {
        printf(COLOR_RED "Action Denied: Current status is 'NEW'.\n" COLOR_RESET);
        printf("Explanation: You cannot skip the 'ASSIGNED' stage.\n");
        printf("Instruction: Please use Option 3 (Assign Technician) to assign a technician
first.\n");
    }
    else if (strcmp(requests[idx].status, "ASSIGNED") == 0) {
        char confirm;
        printf("This request is currently assigned to Technician ID %d.\n",
requests[idx].techId);
        printf("Is the maintenance job fully completed? (y/n): ");
        scanf("%c", &confirm);
        clearInputBuffer();

        if (confirm == 'y' || confirm == 'Y') {
            strcpy(requests[idx].status, "DONE");
            printf(COLOR_GREEN "Success: Status updated to 'DONE'. Job Closed.\n"
COLOR_RESET);
```

```
        } else printf("Update cancelled. Status remains 'ASSIGNED'.\n");
    }
    else if (strcmp(requests[idx].status, "DONE") == 0) {
        printf(COLOR_GREEN "Information: This request is already archived as 'DONE'.\n"
COLOR_RESET);
        printf("No further status changes are permitted.\n");
    }
    else printf(COLOR_RED "System Error: Invalid or corrupted status detected.\n"
COLOR_RESET);
}

// [Page 4, 2.5] Search Technician
void searchTechnician() {
    int techId;
    printf("Enter Technician ID to search: ");
    scanf("%d", &techId);
    clearInputBuffer();
    int idx = findTechnicianById(techId);

    if (idx != -1) {
        printf(COLOR_GREEN "\n--- Technician Found ---\n" COLOR_RESET);
        printf("ID: %d\nName: %s\nSpecialty: %s\n",
               technicians[idx].techId, technicians[idx].name, technicians[idx].specialty);
    } else printf(COLOR_RED "Technician not found.\n" COLOR_RESET);
}

// Statistics Dashboard (Creativity)
void showStatistics() {
    int newC = 0, assignedC = 0, doneC = 0;
    for(int i=0; i<requestCount; i++) {
        if(strcmp(requests[i].status, "NEW") == 0) newC++;
        else if(strcmp(requests[i].status, "ASSIGNED") == 0) assignedC++;
        else if(strcmp(requests[i].status, "DONE") == 0) doneC++;
    }

    printf(COLOR_CYAN "\n=== SYSTEM STATISTICS ===\n" COLOR_RESET);
    printf("Total Requests: %d\n", requestCount);
    printf("NEW:      %d\n", newC);
    printf("ASSIGNED: %d\n", assignedC);
    printf("DONE:     %d\n", doneC);
    printf("-------------------------\n");
}

// [Page 4, 2.6] Export NEW Requests to a File
void generateReport() {
    FILE *fp = fopen(FILENAME, "w");
    if (fp == NULL) {
```

```
        printf(COLOR_RED "Critical Error: Could not open file '%s' for writing.\n"
COLOR_RESET, FILENAME);
        printf("Check if you have write permissions or if the file is open elsewhere.\n");
        return;
    }

    int count = 0;
    // Write Header
    fprintf(fp, "--- Campus Maintenance New Requests Report ---\n");
    fprintf(fp, "Generated on: 2025/09 Session\n");
    fprintf(fp, "ID,Location,Category,Priority\n");

    printf(COLOR_CYAN "\n--- Generating Summary Report ---\n" COLOR_RESET);
    printf("Scanning database for 'NEW' requests...\n");

    for (int i = 0; i < requestCount; i++) {
        if (strcmp(requests[i].status, "NEW") == 0) {
            // Write to file
            fprintf(fp, "%d,%s,%s,%d\n",
                    requests[i].reqId,
                    requests[i].location,
                    requests[i].category,
                    requests[i].priority);

            //Detailed Output
            printf(" -> Exporting Request ID: %d (Loc: %s)\n", requests[i].reqId,
requests[i].location);
            count++;
        }
    }

    fclose(fp); // Close the file
    // Final detailed success message
    printf(COLOR_CYAN "-------------------------------------------\n" COLOR_RESET);
    if (count > 0) {
        printf(COLOR_GREEN "Success: Report generation complete.\n" COLOR_RESET);
        printf("Total Records Exported: %d\n", count);
        printf("Data saved to file: " COLOR_CYAN "%s" COLOR_RESET "\n", FILENAME);
        printf("You can find this file in the program directory.\n");
    } else {
        printf("Result: No 'NEW' requests found to export.\n");
        printf("The file '%s' has been created but is empty.\n", FILENAME);
    }
}
```